Course Project: Cryptographic Algorithms & ModularArithmetic
Shamir's Secret Sharing

1 Shamir's Secret Sharing and it's purpose

Shamir's Secret Sharing (SSS) algorithm is an encryption algorithm that takes a secret, and split it into different pieces or, in other words, shares with the number of shares is denoted by N. The secret can only be reconstructed if there is enough shares, or meet the threshold of shares, which is denoted by a number K.

Basically, a secret can be split among different people, and those people can only reconstruct the secret if enough people agree to recreate it.

How it can be implemented, polynomials would be used to create the shares and also recreate the secret. Lets say a polynomial $y = ax + b$. The b in this case is the constant, and for SSS it would be the secret that is to be encrypted. Shares would be made by plugging in random numbers of x and finding the y, and that pair (x, y) would be a share. The coefficient(s) can be any random number when creating shares. For decrypting a secret, the polynomial that goes through K points must be found, and again the constant of that polynomial would be the secret. Using the modulus can also be used in SSS, but will be covered later.

2 Mathematical background

In order to implement SSS, there are some concepts to go over first. The first which is the **Lagrange Polynomial Interpolation**, which is used to find the polynomial of the lowest degree that goes through all of the shares. A Lagrange polynomial of some share's x coordinate x_i, is a polynomial that its y value at that x_i is 1, and is 0 for every other share's, at least the shares inputted, x coordinate.

For example, lets say there are some x coordinates $x = 1, 2, 3, 4$, the Lagrange equation of the x coordinate 2, would be some polynomial such that $f(2) = 1$, and $f(1) = f(3) = f(4) = 0$

After getting the polynomial for that value of x_i, the polynomial would be multiplied by the x_i's respective y value, and now there would be an equation that still is 0 at all other share's x value, but now goes through the x_i's share. The same thing will be done for all other shares, where the the Lagrange polynomial for those shares are found, and multiply by their respective y values. After getting all of these Lagrange polynomials and add them together, there would now be a polynomial that goes through all of the shares.

Here is the equations for Lagrange Polynomial Interpolation.

$$l_i(x) = \prod_{j=0, j\neq i}^{n} \frac{x - x_j}{x_i - x_j}$$

$$f(x) = \sum_{i=0}^{K-1} y_i l_i(x)$$

Looking at the first line, there is a product operator ($\prod$). This means that all of the fractions where the divisor is the value of x_i who's Lagrange equation is being found minus another share's x value, and the dividend is x minus the same share's x value as is being subtracted in the divisor, and all of those fractions are multiplied together.

For example, say that, again, there are some x coordinates $x = 1, 2, 3, 4$, the Lagrange equation of $x = 2$ would be:

$$l_2(x) = \frac{x-1}{2-1} \times \frac{x-3}{2-3} \times \frac{x-4}{2-4}$$

Now looking at the second equation, this is the implementation of multiplying each Lagrange polynomial by their respective y, then adding all Lagrange polynomials to finally get the polynomial that goes through all the shares.

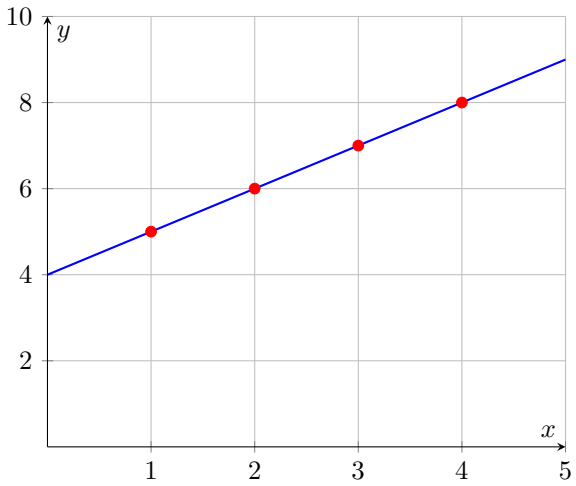
Walking through the full example of $x = 1, 2, 3, 4$, the follow Lagrange polynomials are found:

$$l_1(x) = \frac{x-2}{1-2} \times \frac{x-3}{1-3} \times \frac{x-4}{1-4}$$
$$l_2(x) = \frac{x-1}{2-1} \times \frac{x-3}{2-3} \times \frac{x-4}{2-4}$$
$$l_3(x) = \frac{x-1}{3-1} \times \frac{x-2}{3-2} \times \frac{x-4}{3-4}$$
$$l_4(x) = \frac{x-1}{4-1} \times \frac{x-2}{4-2} \times \frac{x-3}{4-3}$$

Now, multiplying each Lagrange polynomial by there respective y, let it be $y = 5, 6, 7, 8$, then simplifying the equation, the following is found:

$$f(x) = x + 4$$

Plotting the line $f(x) = x + 4$, it does indeed go thought the points $(1, 5), (2, 6), (3, 7), (4, 8)$.

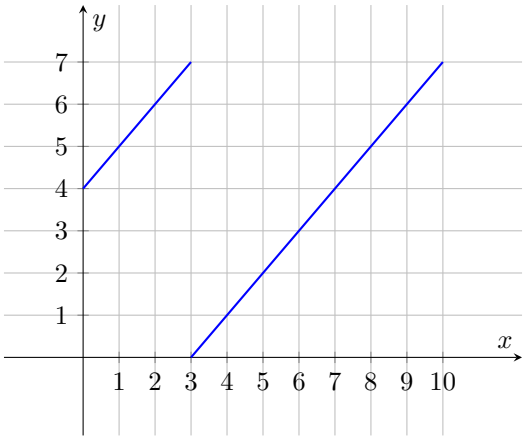


In the case of SSS, we would not use the variable x at all like in the equation $f(x) = x + 4$. Instead, the values of each Lagrange polynomial at $x = 0$ would be looked for, as the constant of the polynomial would be the secret. So the new equations for the use of SSS would be:

$$l_i(0) = \prod_{j=0, j\neq i}^{n} \frac{-x_j}{x_i - x_j}$$
$$f(x) = \sum_{i=0}^{K-1} y_i l_i(0)$$

The bottom equation would stay the same.

With just the Lagrange Polynomial Interpolation, a basic version of SSS can be achieved, but another concept that could be added to improve the security of the algorithm. This concept is **Modular Arithmetic**. Modular Arithmetic is where the remainder of division. For example, the remainder of 8 divided by 6 is 2. Equivalently, $8 \bmod 6$ would be 2. When applying this to an equation, for example $(x + 4) \bmod 7$ we get:



Looking at this graph, we can see that the equation $y = x + 4$ loops around under the line $x = 7$. How does this apply to the SSS algorithm? Now that we have a polynomial with the secret of 4 under the modulus of 7 in this case, we make some shares. Let $x = 1, 2, 3, 4$. Plug them into the polynomial and we get $y = 5, 6, 0, 1$. Now the points look scrambled, and would be much harder to brute force looking for a polynomial that contains the secret using Lagrange if there are some shares known but not enough to recreate the secret. The divisor number thing will be referred to as modulus in this report, like the number 7 in $\mod 7$ will be referred to as modulus.

One more thing to note about doing $\mod$ is that we cannot do regular division under $\mod$. For example, the fraction in $(\frac{3}{7}) \mod 5$ cannot be done regularly. The way around this is that we have to find the **mod inverse** of the divisor. The $\mod$ inverse of a number a is a number b under $\mod c$ such that $b \cdot a \mod c = 1$. For example, the $\mod$ inverse of 7 under $\mod 5$ is 3 since $3 \cdot 7 \mod 5 = 1$. Now that we have the $\mod$ inverse, we can multiply the dividend by the $\mod$ inverse and that would be how we do division under $\mod$.

3 Pseudocode

Now how can we implement all of this? Let us first look at encryption. To encrypt a secret, we need the number that we want to be a secret S, the number of shares N, and the threshold K, making sure that $N \geq K$ since we need enough shares to recreate the secret. The degree of polynomial needed to encrypt the secret would be $K - 1$, so for threshold of 3, we would need a second degree polynomial. for example $ax^2 + bx + c = y$. The coefficients can be any random integer, but let us make sure that all of them are not 0. The secret would again be the constant c. Now to implement the $\mod$ stuff, we must choose a prime number to be the modulus to avoid any weirdness when doing the $\mod$ calculation. The modulus must also be greater than S, K, N. For this report, we will choose the next biggest prime number of the greatest number between the S, K, N. Now lets get the steps:

1. Input the S, K, N, making sure that $N \geq K$
2. Calculate what the

Algorithm 1 Find Maximum in Array

```
1: procedure FindMax(A, n)
2:     max ← A[1]
3:     for i ← 2 to n do
4:         if A[i] > max then
5:             max ← A[i]
6:         end if
7:     end for
8:     return max
9: end procedure
```